

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS REITOR JOÃO DAVID FERREIRA LIMA
CURSO DE SISTEMAS DE INFORMAÇÃO

Guilherme Focassio dos Santos

ANÁLISE E IMPLEMENTAÇÃO DE UM MOTOR DE SINCRONIZAÇÃO DESCENTRALIZADA PARA APLICAÇÕES EM TEMPO REAL

Florianópolis, Santa Catarina, Brasil
2027

Guilherme Focassio dos Santos

ANÁLISE E IMPLEMENTAÇÃO DE UM MOTOR DE SINCRONIZAÇÃO DESCENTRALIZADA PARA APLICAÇÕES EM TEMPO REAL

Trabalho de Conclusão de Curso submetido ao Curso de Sistemas de Informação da Universidade Federal de Santa Catarina para a obtenção do Grau de Bacharel em Sistemas de Informação.
Orientador(a): Odorico Machado Mendizabal, Dr.

Florianópolis, Santa Catarina, Brasil
2027

Notas legais:

Não há garantia para qualquer parte do software documentado. Os autores tomaram cuidado na preparação desta tese, mas não fazem nenhuma garantia expressa ou implícita de qualquer tipo e não assumem qualquer responsabilidade por erros ou omissões. Não se assume qualquer responsabilidade por danos incidentais ou consequentes em conexão ou decorrentes do uso das informações ou programas aqui contidos.

Catalogação na fonte pela Biblioteca Universitária da Universidade Federal de Santa Catarina.
Arquivo compilado às 09:42h do dia 27 de outubro de 2026.

Guilherme Focassio dos Santos
Análise e Implementação de um Motor de Sincronização Descentralizada para Aplicações em Tempo Real / Guilherme Focassio dos Santos; Orientador(a), Odorico Machado Mendizabal, Dr. – Florianópolis, Santa Catarina, Brasil, [dia] de [mês] de 2027.
44 p.

Trabalho de Conclusão de Curso – Universidade Federal de Santa Catarina, INE – Departamento de Informática e Estatística, CTC – Centro Tecnológico, Curso de Sistemas de Informação.

Inclui referências

1. Peer-to-Peer, 2. P2P, 3. Cliente-Servidor, 4. CRDTs, 5. Sincronização de Estados, I. Odorico Machado Mendizabal, Dr. II. Curso de Sistemas de Informação III. Análise e Implementação de um Motor de Sincronização Descentralizada para Aplicações em Tempo Real

CDU 02:141:005.7

Guilherme Focassio dos Santos

ANÁLISE E IMPLEMENTAÇÃO DE UM MOTOR DE SINCRONIZAÇÃO DESCENTRALIZADA PARA APLICAÇÕES EM TEMPO REAL

Este(a) Trabalho de Conclusão de Curso foi julgado adequado(a) para obtenção do Título de Bacharel em Sistemas de Informação, e foi aprovado em sua forma final pelo Curso de Sistemas de Informação do INE – Departamento de Informática e Estatística, CTC – Centro Tecnológico da Universidade Federal de Santa Catarina.

Florianópolis, Santa Catarina, Brasil, [dia] de [mês] de 2027.

Nome do Coordenador(a), Dr.
Coordenador(a) do Curso de Sistemas de Informação

Banca Examinadora:

Odorico Machado Mendizabal, Dr.
Orientador(a)
Universidade Federal de Santa Catarina – UFSC

Prof. Convidado 1, Dr.
Instituição 1 – Sigla 1

Prof. Convidado 2, Dr.
Instituição 2 – Sigla 2

Prof. Convidado 3, Dr.
Instituição 3 – Sigla 3

Prof. Convidado 4, Dr.
Instituição 4 – Sigla 4

Dedico esse trabalho ao meu eu do passado, que sempre sonhou em trazer uma vida melhor para as pessoas do seu entorno. Também gostaria de fazer uma saudação ao meu do futuro, que conquistou todos os objetivos que almejamos. Este trabalho de conclusão de curso não foi o primeiro, e, certamente, não será o nosso último passo.

AGRADECIMENTOS

[Escreva aqui seus agradecimentos.]

“Nature builds no machines, no locomotives, railways, electric telegraphs, self-acting mules etc. [. . .] These are organs of the human brain, created by the human hand; the power of knowledge, objectified.”

(MARX, 1973, p. 706)

RESUMO

As arquiteturas de redes de computadores sempre tiveram um forte envolvimento com a velocidade e escalabilidade das soluções distribuídas desenvolvidas por profissionais para problemas em tempo real. Atualmente, sistemas modernos distribuídos enfrentam limitações de escalabilidade, latência e resiliência decorrentes da dependência em arquiteturas cliente-servidor centralizadas. As limitações crescem quando o assunto são sistemas distribuídos interativos em tempo real, ou seja, com usuários ativamente mudando estados, fazendo trocas e atualizações que devem ser sincronizadas com o servidor central. Como uma alternativa a essas propostas, arquiteturas Peer-to-Peer (P2P) surgiram com resoluções e novos desafios. Elas descentralizam o estado, para a carga não ficar sobrecarregada em um só nodo (servidor), mas introduzem complexidades relacionadas a concorrência e divergência de dados. Para avaliarmos o uso e os possíveis endereçamentos de suas nuances será feito um PoC (Proof of Concept) utilizando mapas para sincronização de dados entre múltiplos usuários em tempo real.

Palavras-chaves: Peer-to-Peer. P2P. Cliente-Servidor. CRDTs. Sincronização de Estados.

ABSTRACT

This is the English abstract.

Keywords: Keyword 1. Keyword 2. Keyword 3. Keyword 4. Keyword 5.

LISTA DE TABELAS

Tabela 1 – Comparativo entre libp2p e Iroh 39
Tabela 2 – Comparativo entre Yjs e Automerge 40

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
abnTeX	ABsurdas Normas para TeX

LISTA DE SÍMBOLOS

Γ	Letra grega Gama
Λ	Lambda
ζ	Letra grega minúscula zeta
$\in$	Pertence

SUMÁRIO

I	**PESQUISA**	**14**
1	**INTRODUÇÃO**	**15**
1.1	OBJETIVOS	15
1.1.1	**Objetivos gerais**	**16**
1.1.2	**Objetivos específicos**	**16**
1.2	ESCOPO DO TRABALHO	17
1.3	MÉTODO DE PESQUISA	17
1.4	O USO DE INTELIGÊNCIA ARTIFICIAL NESTE TRABALHO	17
1.5	ORGANIZAÇÃO DO TRABALHO	18
2	**FUNDAMENTAÇÃO TEÓRICA**	**19**
2.1	EVOLUÇÃO HISTÓRICA DAS ARQUITETURAS DISTRIBUÍDAS E DO MODELO P2P	19
2.1.1	**As Origens na ARPANET e a Simetria Original**	**19**
2.1.2	**A Dominância do Modelo Cliente-Servidor e a Crise de Escalabilidade**	**19**
2.1.3	**P2P de Primeira Geração: A Arquitetura Híbrida do Napster**	**20**
2.1.4	**P2P de Segunda Geração: Descentralização Não Estruturada, *Flooding* e Supernós**	**21**
2.1.5	**P2P de Terceira Geração: Redes Estruturadas, BitTorrent e DHTs (Kademlia)**	**22**
2.2	SINCRONIZAÇÃO DE ESTADO MUTÁVEL EM TEMPO REAL	23
2.2.1	**A Limitação dos Dados Estáticos em Redes P2P Tradicionais**	**23**
2.2.2	**Ausência de Relógio Global, Causalidade, Relógios Lógicos e Vetoriais**	**24**
2.2.3	**O Dilema da Concorrência e Consistência Eventual Forte (SEC)**	**25**
2.3	ESTRUTURAS DE DADOS PARA RESOLUÇÃO DESCENTRALIZADA DE CONFLITOS (CRDTS)	26
2.3.1	**Semirreticulados Superiores (*Join-Semilattices*) e Propriedades Matemáticas**	**26**
2.3.2	**Abordagens de CRDTs: *State-based* (CvRDT), *Operation-based* (CmRDT) e *Delta-State***	**27**
3	**ESTADO DA ARTE E TRABALHOS RELACIONADOS**	**29**
3.1	O PARADIGMA *LOCAL-FIRST SOFTWARE* E A SOBERANIA DOS DADOS	29

3.2 *BLOCKCHAIN* E WEB3: CONSENSO GLOBAL VERSUS CONVERGÊNCIA SEM CONSENSO . . . 30
3.3 PROTOCOLOS SOCIAIS DESCENTRALIZADOS: AT PROTOCOL E NOSTR . . . 30
3.3.1 Nostr: Notes and Other Stuff Transmitted by Relays . . . 31
3.3.2 AT Protocol: Authenticated Transfer Protocol (Bluesky) . . . 31
3.4 LIBP2P: A MODULAR NETWORKING STACK FOR PEER-TO-PEER APPLICATIONS . . . 32
3.4.1 Contexto e motivação . . . 32
3.4.2 Funcionamento . . . 32
3.4.3 Implementações e Casos de Uso . . . 33
3.5 IROH . . . 33
3.5.1 Contexto e motivação . . . 33
3.5.2 Funcionamento . . . 33
3.5.3 Implementações e Casos de Uso . . . 34
3.6 YJS . . . 34
3.6.1 Contexto e motivação . . . 34
3.6.2 Funcionamento . . . 35
3.6.3 Implementações e Casos de Uso . . . 35
3.7 AUTOMERGE . . . 36
3.7.1 Contexto e motivação . . . 36
3.7.2 Funcionamento . . . 36
3.7.3 Implementações e Casos de Uso . . . 37
3.8 ORBITDB E MERKLE-CRDTS . . . 37
3.8.1 Contexto e motivação . . . 37
3.8.2 Funcionamento . . . 38
3.8.3 Implementações e Casos de Uso . . . 38
3.9 SINCRONIZAÇÃO COLABORATIVA DE DADOS ESPACIAIS (WEBGIS) . . . 39
3.10 COMPARATIVO E POSICIONAMENTO DO TRABALHO . . . 39

4 CONTEXTO DO ESTUDO DE CASO E CONCEITOS . . . 41

5 ARQUITETURA E DESENVOLVIMENTO DO MOTOR DE SINCRONIZAÇÃO . . . 42

6 AVALIAÇÃO EXPERIMENTAL E RESULTADOS . . . 43

7 CONCLUSÃO . . . 44

Parte I

Pesquisa

1 INTRODUÇÃO

As redes Peer-to-peer estão na computação como conceito e aplicações desde os primórdios da ARPANET. Na verdade, a própria ARPANET poderia ser descrita como uma rede de comunicação entre nodos - ou “pares” - descentralizada (Crocker, 1969), o que se refletiu na própria criação da internet. A internet como conhecemos hoje é uma comunicação entre múltiplas máquinas, múltiplos pares, para envio e recebimento de recursos. Longe de ser uma novidade recente, o conceito é maduro e fundamenta a comunicação entre sistemas desde a década de 1960.

O conceito evoluiu ao longo de três grandes gerações de redes peer-to-peer, sempre adicionando ou melhorando a solução anterior. Na primeira geração houve a primeira grande implementação dessa arquitetura para resolver o problema de transferência de mídias digitais, ainda com servidores centrais direcionando o tráfego com tabelas de índices. A segunda geração verdadeiramente descentralizou a rede, não dependendo mais de nenhum servidor central. Então, a terceira geração de redes eliminou os principais problemas de flooding, introduziu o upload em chunks e muitos outros problemas de suas antecessoras, se tornando um sucesso global.

Entretanto, essa evolução se restringiu ao envio de arquivos estáticos. Arquivos nunca mudam entre um envio e outro, uma música sempre será o mesmo file, terá o mesmo hash, os mesmos metadados. Para as aplicações do mundo moderno, pautadas no envio e recebimento massivo de dados de aplicações em tempo real, as redes peer-to-peer enfrentam um novo grande desafio: a sincronização de dados em tempo real.

Como uma alternativa para superar essas limitações sem recorrer ao modelo cliente-servidor, este trabalho propõe o desenvolvimento e análise de um Motor de Sincronização Descentralizada para Aplicações em Tempo Real. Para fundamentar e resolver o conflito de divergência sem autoridade central, o motor utiliza estruturas de dados específicas para consistência eventual, como Conflict-free Replicated Data Types (CRDTs) (**shapiro2011**). Para validar e avaliar o desempenho da solução em termos de latência, resiliência e concorrência, é desenvolvida uma Proof of Concept (PoC) focada na sincronização espacial de mapas interativos entre múltiplos usuários simultâneos.

1.1 OBJETIVOS

Esta seção busca formalizar os objetivos do presente trabalho.

1.1.1 Objetivos gerais

Este trabalho tem como objetivo projetar, implementar e avaliar experimentalmente um motor de sincronização de estado descentralizado (Peer-to-Peer), de baixo nível e agnóstico a domínio, visando garantir a Consistência Eventual Forte (SEC) em aplicações de tempo real por meio da integração de Tipos de Dados Replicados Livres de Conflitos (CRDTs) e ordenação lógica de eventos.

1.1.2 Objetivos específicos

- Estudar os fundamentos teóricos de consistência em sistemas distribuídos descentralizados, analisando os conceitos de Consistência Eventual Forte (SEC), Tipos de Dados Replicados Livres de Conflitos (CRDTs), relógios lógicos e protocolos de fofoca (gossip);
- Analisar os requisitos técnicos e desafios práticos para o desenvolvimento de sistemas de alta performance em tempo real, mapeando os requisitos de paralelismo, controle de concorrência, latência e segurança no ciclo de vida da memória;
- Projetar a arquitetura modular de baixo nível do motor de sincronização, definindo os fluxos de entrada e saída de rede assíncrona, enquadramento (framing) de dados, controle de concorrência multi-thread e o modelo de persistência volátil;
- Implementar o middleware (motor de sincronização) focando na performance de execução, empregando mecanismos eficientes de serialização de dados (zero-copy) e estruturas de dados otimizadas para manipulação concorrente;
- Desenvolver um protótipo funcional de tempo real (aplicação de prova de conceito) que consuma as interfaces expostas pelo motor, demonstrando na prática o comportamento dinâmico e a natureza agnóstica do sistema;
- Avaliar a solução proposta por meio de testes experimentais, analisando métricas de tempo de convergência de réplicas, taxa de transferência de rede (payload de fofoca), latência induzida e consumo de recursos de processamento e memória;
- Documentar as limitações de design identificadas, os aprendizados de engenharia de software aplicados a sistemas distribuídos e as futuras direções para evolução da infraestrutura proposta.

1.2 ESCOPO DO TRABALHO

1.3 MÉTODO DE PESQUISA

Este trabalho utiliza uma pesquisa aplicada, com foco em investigar um problema prático da engenharia de software distribuída: a viabilidade e os desafios de manter estados consistentes em redes de tempo real descentralizadas sem coordenação síncrona global. A abordagem adotada é experimental, baseada no projeto, desenvolvimento e avaliação empírica de um protótipo funcional.

Inicialmente, será realizado um estudo teórico aprofundado a respeito das bases matemáticas de CRDTs (com foco em semilattices superiores e mutações baseadas em delta), relógios de Lamport, relógios vetoriais e algoritmos de disseminação epidêmica (gossip protocols). Além disso, serão revisadas as bibliotecas e técnicas modernas de programação de sockets de rede assíncronos e de alta concorrência na linguagem de programação que será adotada.

Em seguida, será modelada e documentada a arquitetura lógica do motor de sincronização. Esse projeto envolverá o mapeamento das dependências entre os módulos de rede (camada de transporte), de persistência do estado da réplica, de ordenação lógica de eventos e de difusão de atualizações.

Após a etapa de projeto arquitetural, será desenvolvido o motor de sincronização (middleware) sob uma estrutura multi-thread otimizada, implementando as classes e interfaces correspondentes aos CRDTs e relógios lógicos de forma genérica. Concomitantemente, será desenvolvida uma aplicação de prova de conceito (como um simulador distribuído simples ou um quadro colaborativo interativo) cuja única responsabilidade será manipular o estado através do middleware, validando sua natureza agnóstica a domínio.

Por fim, serão conduzidos testes práticos e ensaios experimentais sob diferentes cenários de carga de trabalho e concorrência induzida. Esses ensaios buscarão caracterizar quantitativamente a performance da solução proposta, avaliando as métricas de tempo total de convergência do estado de todas as réplicas ativas, taxa de pacotes descartados por falta de causalidade, consumo de banda de rede (bits/segundo) e consumo de processador e memória RAM nos nós da rede simulada. Os resultados obtidos guiarão a redação das conclusões e limitações técnicas do sistema.

1.4 O USO DE INTELIGÊNCIA ARTIFICIAL NESTE TRABALHO

Em conformidade com as boas práticas de transparência acadêmica, esclarece-se nesta seção como ferramentas de Inteligência Artificial (IA) foram empregadas na elaboração deste trabalho.

Um assistente de IA (Claude, da Anthropic, operando através da ferramenta Claude Code) foi utilizado como apoio em tarefas de natureza instrumental e editorial,

e não na concepção científica da pesquisa. Especificamente, a ferramenta auxiliou:

- Na migração do texto deste documento, originalmente redigido pelo autor em processador de texto, para o template LaTeX da UFSC em conformidade com as normas ABNT, preservando o conteúdo e a redação originais do autor;
- Na configuração do ambiente de desenvolvimento LaTeX local (distribuição TeX Live, editor, extensões) e do fluxo de versionamento do projeto via `git`;
- Na formatação das referências bibliográficas citadas neste capítulo no formato BibTeX e na inserção dos respectivos comandos de citação no texto;
- Na organização geral do repositório do projeto, incluindo a remoção de conteúdo de demonstração herdado do template original.

A definição do tema de pesquisa, a revisão da literatura, a redação do conteúdo técnico e científico, as decisões metodológicas e as conclusões apresentadas ao longo deste trabalho são de autoria e responsabilidade do autor. O uso da IA restringiu-se, portanto, a um papel de ferramenta de produtividade na produção do documento, análogo ao uso de um editor de texto ou de um verificador ortográfico, sem substituir o julgamento científico ou a autoria intelectual da pesquisa.

1.5 ORGANIZAÇÃO DO TRABALHO

Além desta introdução, o restante deste trabalho está organizado da seguinte forma: o capítulo de Desenvolvimento apresenta a fundamentação teórica sobre consistência em sistemas distribuídos descentralizados – CRDTs, relógios lógicos e protocolos de disseminação epidêmica – seguida do projeto da arquitetura modular do motor de sincronização e dos detalhes de sua implementação, com foco em desempenho e concorrência. Em seguida, é apresentado o protótipo de prova de conceito desenvolvido sobre o motor, e a avaliação experimental conduzida sobre ele, caracterizando métricas de convergência, latência, tráfego de rede e consumo de recursos sob diferentes cenários de carga. Por fim, o capítulo de Conclusão retoma os objetivos definidos neste capítulo, discute os resultados obtidos e as limitações identificadas, e aponta direções para trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 EVOLUÇÃO HISTÓRICA DAS ARQUITETURAS DISTRIBUÍDAS E DO MODELO P2P

2.1.1 As Origens na ARPANET e a Simetria Original

As redes *Peer-to-peer* estão na computação como conceito e aplicações desde os primórdios da ARPANET. Na verdade, a própria ARPANET poderia ser descrita como uma rede de comunicação entre nodos – ou "pares" – descentralizada (**crocker1969**), o que se refletiu na própria criação da internet. A internet como é conhecida hoje é uma comunicação entre múltiplas máquinas, múltiplos pares, para envio e recebimento de recursos. Longe de ser uma novidade recente, o conceito é maduro e fundamenta a comunicação entre sistemas desde a década de 1960.

2.1.2 A Dominância do Modelo Cliente-Servidor e a Crise de Escalabilidade

Nos anos 90 ocorreu uma transição do que se conhecia anteriormente como ARPANET para a World Wide Web, trazendo com a mudança milhões de usuários leigos buscando e consumindo informações de uma maneira nunca antes vista na história da humanidade. Nesse aspecto, um modelo vigorou, dominou e se consolidou como o padrão desse novo ambiente: o modelo cliente-servidor (**fieldingtaylor2000**; **tanenbaumvansteen2007**). Para entender os principais motivos do sucesso desse novo modelo, é preciso repensar algumas características das redes *peer-to-peer* que precisariam ser adaptadas para essa nova realidade.

Redes *peer-to-peer* trocam mensagens para definir qual a versão "correta" – muitas vezes subjetiva – de um pacote, dado ou informação. Com o advento da internet, milhões de novas máquinas começaram a processar, enviar e receber esses pacotes/informações a todo momento. Isso, na época, foi um grande problema relacionado à descoberta e busca de recursos entre os nodos, saber onde o recurso ou arquivo estava para fazer o upload ou download (**ripeanu2001**).

Naquela época, muitas residências tinham altas – para a época – taxas de download, mas baixíssimas taxas de upload. Fazer todas essas máquinas, algumas vezes rápidas, outras lentas, responderem requisições teria uma latência e velocidade muito inferior a simplesmente um *data center* com links dedicados de alta velocidade, capazes de responder milhares de clientes passivos (**saroiu2002**).

Somado a isso, os computadores pessoais não precisavam de IP público estático, portas abertas e nem processar continuamente, garantindo a simplicidade do modelo no lado do cliente (**saltzer1984**).

A adoção do modelo cliente-servidor viabilizou a difusão rápida da Web em seus primeiros anos. Entretanto, no final da década de 1990, uma nova tendência surgiu: a

explosão de mídias digitais, impulsionada pela popularização dos arquivos de áudio em MP3. A procura e compartilhamento de músicas se espalhou e trouxe consigo um aumento no número de bytes trafegados. A rede não estava mais apenas baixando e fazendo upload de HTML e arquivos de textos de poucos kbytes, os arquivos de músicas elevaram esse patamar para os megabytes, causando a explosão das contas das infraestruturas dos servidores centrais (**saroiu2002**). Para ilustrar esse gargalo de largura de banda (*egress bandwidth*), em um cenário hipotético no qual um milhão de usuários querem baixar um HTML de 20kbytes o servidor precisará lidar com cerca de 20 Gigabytes de dados de saída, com um arquivo MP3 de 5MB no mesmo servidor, seria necessária uma demanda massiva de 5 Terabytes, um volume desproporcional para a infraestrutura da época.

O volume de dados massivo forçava os servidores centrais a um limite o qual eles não eram capazes de suportar sem encarecer demais o serviço. A centralização do estado e dos recursos fez com que a disponibilidade de todo o sistema ficasse refém do servidor central, um único ponto de falha (*Single Point of Failure*, ou SPOF) (**tanenbaumvansteen2007**). Isso gerou *flash crowds* (picos de tráfego) que derrubaram servidores inteiros por exaustão de CPU, indisponibilizando o serviço para todos os usuários (**padmanabhan2002**).

Os pontos negativos do modelo cliente-servidor vieram à tona, escalabilidade, disponibilidade, custo e manutenção eram apenas alguns deles. A crise criou o cenário perfeito para o ressurgimento das redes *Peer-to-peer*, que reemergiam como uma alternativa viável e escalável para o problema, em 1999 surge o Napster.

2.1.3 P2P de Primeira Geração: A Arquitetura Híbrida do Napster

Criado em 1999 por Shawn Fanning e Sean Parker, o Napster como solução de software provou a viabilidade de uma implementação P2P em larga escala (**saroiu2002**). Sua implementação foi realizada para resolver um problema comum da época: a dificuldade de baixar arquivos mp3, como músicas, de sites estáticos ou canais de IRC da época.

O Napster resolveu o gargalo financeiro e o de largura de banda do modelo cliente-servidor na distribuição desses arquivos, na época, pesados. Por exemplo, para uma empresa distribuir um vídeo de 1 Terabyte de dados, seria necessário 1 Terabyte de *egress bandwidth* dos *data centers*. Com o Napster essa infraestrutura de rede, hardware e conexão é repassada para os próprios usuários da rede, os nodos ou *peers*. Os usuários que querem baixar um determinado arquivo fazem essa busca pela rede até encontrar alguém o servindo. A partir disso, cria-se uma conexão entre os dois, onde os recursos computacionais de ambos são usados para o compartilhamento, reduzindo, e muito, o estresse das máquinas centrais dos *data centers* (**adarhuberman2000**).

Mesmo com tudo isso, o Napster ainda não era uma rede totalmente descen-

tralizada. Em vez disso, atuava como uma arquitetura híbrida, ou P2P de primeira geração, combinando elementos de centralização com transferência direta de dados (**schollmeier2001**). Na execução do programa, o cliente se conecta com servidores centrais mantidos pelo Napster. O cliente, então, envia a lista de todos os MP3 armazenados para compartilhamento, endereço IP e porta de conexão. Os servidores centrais do Napster mantinham em memória índices globais e dinâmicos em tempo real, mapeando milhões de arquivos com respectivos endereços de IP. Quando um usuário procurava por uma música, pelo nome, por exemplo, essa pesquisa era convertida em uma *query* e enviada para o servidor central. O servidor pesquisava em suas tabelas pelo índice e retornava uma lista contando IPs de usuários que estavam provendo aquele conteúdo e suas velocidades de conexão (upload e download). Assim que o usuário escolhia a fonte, a conexão entre ele e o servidor central acabava e começava a conexão TCP direta entre os nodos/*peers*. O arquivo requerido pelo usuário consumidor era, então, enviado do usuário provedor para o seu computador. Embora o problema do gargalo tenha sido resolvido, dois pontos fracos principais finalizaram as operações do Napster em 2001:

- Caso a infraestrutura dos servidores centrais, detentores das tabelas de índices, fosse derrubada, toda a rede como um todo parava de funcionar. Algo análogo a um motorista em uma cidade desconhecida com GPS desligado. Os servidores centrais eram o novo ponto de falha única (SPOF – *Single point of failure*) do sistema (**tanenbaumvansteen2007**);
- Esse ponto único de falha foi justamente o foco principal do ataque contra o Napster realizado por cortes e juízes americanos, que definiram a derrubada desses servidores. Com os servidores fora do ar, a rede perdia o seu propósito.

2.1.4 P2P de Segunda Geração: Descentralização Não Estruturada, *Flooding* e Supernós

Após a queda do Napster, uma rede P2P de primeira geração (híbridas com servidor central), como ficou conhecida posteriormente, surgiram protocolos 100% descentralizados e não estruturados, como o Gnutella (**ripeanu2001**). Redes P2P de segunda geração são marcadas, principalmente, pela eliminação do servidor central usando *flooding* de mensagens, não existia servidor central nem tabela de índice. Quando um *peer* quer um arquivo, ele envia mensagens diretamente para seus nodos vizinhos, que fazem o mesmo, consecutivamente, até o nodo detentor do arquivo ser encontrado. Uma vez encontrado, uma conexão direta é estabelecida entre ambos para upload ou download do arquivo, a depender da ponta. O claro gargalo é o *flooding*, ou inundação, de mensagens na rede, gerando um tráfego exponencial $O(d^k)$, onde d é o número de conexões e k o TTL. Em uma rede com milhões de nós a internet dos

usuários poderia ser consumida apenas com tráfego de terceiros.

Almejando o controle do *flooding*, foram introduzidos o conceito de super nós, uma arquitetura semi-híbrida (**saroiu2002**; **yanggarciamolina2003**; **liang2006**). Esses super nós faziam a busca (o *flooding*) entre eles, que possuíam muitos IPs dinâmicos usados para direcionar o fluxo otimizado da mensagem. Como exemplo, pode-se imaginar uma carta enviada pelos correios. Em vez de cada vizinho perguntar para os do lado se aquela é a casa correta, apenas as primeiras casas de cada rua se comunicam, e elas sabem quais casas têm em suas ruas. No momento em que uma dessas primeiras casas avisa que conhece a casa alvo, a primeira casa conta diretamente o endereço dela para o envio da carta, criando-se um caminho muito mais direto até achar a casa correta.

2.1.5 P2P de Terceira Geração: Redes Estruturadas, BitTorrent e DHTs (Kademlia)

As redes P2P de terceira geração, criadas por Bram Cohen em 2001, foram popularizadas por clientes de soluções como utorrent (**cohen2003**). O bittorrent revolucionou a transferência de dados ao abandonar o modelo vigente, o qual enviava arquivos inteiros de um nodo a outro. A fragmentação em blocos, ou *pieces*, é definida pela divisão em pequenos blocos, um usuário não precisa esperar o download terminar para começar a enviar dados. Enquanto baixa o bloco B de usuário 1, envia o bloco A que já detém para o usuário 2.

A verdadeira descentralização do bittorrent veio à tona somente com o surgimento das DHTs, ou *distributed hash tables*, em especial o protocolo Kademlia (**maymounkovmazieres2002**), que permitiu a operação sem rastreamento do bittorrent. Em vez de depender de super nós ou inundações cegas, o Kademlia organiza a rede atribuindo um identificador numérico único (*hash* de 160 bits) tanto para os nós quanto para os arquivos. A localização de um dado é feita calculando uma "distância lógica" matemática entre esses identificadores por meio da operação XOR. Para visualizar o funcionamento de uma DHT, pode-se imaginar um grande catálogo telefônico global dividido em seções. Um usuário não precisa guardar o contato de milhões de pessoas na internet; ele guarda apenas uma lista telefônica reduzida, contendo contatos próximos do seu próprio número e alguns contatos estratégicos para regiões mais distantes da rede. Quando deseja buscar um arquivo específico, o nó envia a requisição para o contato de sua tabela que possui a numeração mais próxima da chave do arquivo desejado. Esse contato, por sua vez, redireciona o pedido para alguém ainda mais próximo, afunilando a busca rapidamente até alcançar o detentor do dado. Esse roteamento estruturado garante que a busca por um arquivo em uma rede com milhões de nós seja concluída em apenas $O(\log N)$ saltos de mensagens, eliminando completamente a tempestade de tráfego (*flooding*) característica das redes

de segunda geração.

2.2 SINCRONIZAÇÃO DE ESTADO MUTÁVEL EM TEMPO REAL

2.2.1 A Limitação dos Dados Estáticos em Redes P2P Tradicionais

As *distributed hash tables* na terceira geração das redes P2P foram um incontestável sucesso. A distribuição massiva de dados descentralizados em blocos entre múltiplos milhões de usuários foi alcançada depois de inúmeros avanços e superações de problemas relacionados a hardware, SO e redes de computadores (**maymounkovmazieres2002**; **cohen2003**). Entretanto, essas grandes redes foram projetadas sob a premissa de uma imutabilidade de dados, um arquivo MP3, ou um vídeo, não mudam de tempos em tempos, seu *hash* sempre será aquele, a depender do algoritmo utilizado. Um outro *hash* representa um arquivo completamente diferente, sempre.

As redes de terceira geração, embora muito bem-sucedidas e evoluídas para a época, não foram projetadas para lidar com dados tão mutáveis. Elas utilizavam um modelo mental chamado de *Single-Writer / Multi-Reader* (SWMR), ou seja, um único escritor e múltiplos leitores. Um arquivo MP3, por exemplo, é escrito pelo usuário que o possui, e todos os outros usuários apenas leem esse arquivo, a rede P2P precisa resolver apenas o problema da distribuição eficiente de leitura (*Multi-Reader*). No início dos anos 2000, grandes empresas, como a Google, desenvolveram aplicações web colaborativas, os quais não poderiam ser regidos pelo modelo SWMR, pois múltiplos usuários poderiam alterar o mesmo dado ao mesmo tempo. Esses editores operavam sob o modelo *Multi-Writer / Multi-Reader* (MWMR).

A Web 2.0 precisou, então, resolver o problema da edição colaborativa em tempo real no fim dos anos 2000, impulsionada pelo Google Docs e Etherpad. A indústria respondeu adotando o paradigma *Operational Transformation*, OT (**ellisgibbs1989**). Em vez de enviar um dado por inteiro, o OT envia apenas as operações realizadas sobre o dado. Quando elas ocorrem concorrentemente, o OT passa as operações por uma função matemática de transformação $T(O_1, O_2)$, que ajusta as posições e índices da segunda operação para que ela faça sentido com base no fato de que O_1 já existia (**sunellis1998**).

O OT foi implementado em arquiteturas centralizadas, pois era onde as grandes empresas detinham suas soluções para ambientes colaborativos (**nichols1995**). Contudo, ele se torna complexo e quebradiço em redes P2P descentralizadas.

- A dependência de ordenação global síncrona, torna extremamente difícil e pesado determinar o estado correto das operações concorrentes, exigindo algoritmos de consenso distribuídos.

- A explosão de combinação de estados em uma P2P com múltiplos nós, exige que cada nó implemente propriedades matemáticas, conhecidas como *Transformation Properties* (**ressel1996**). Manter essas propriedades para cenários com três ou mais nós sem uma autoridade central provou ser cientificamente e praticamente inviável, levando a falhas onde o texto ou estado final divergia permanentemente entre as réplicas (**imine2003**; **oster2006**).

- O alto custo de memória histórica, pois cada nó precisa manter um histórico de todas as operações realizadas, para que possa aplicar as transformações corretas em caso de divergência. Inviabilizando o uso para execução nos computadores pessoais e celulares da época.

2.2.2 Ausência de Relógio Global, Causalidade, Relógios Lógicos e Vetoriais

A subseção anterior descreve o problema de divergência de estados concorrentes, não sendo possível saber qual evento foi realizado primeiro, ou se eles ocorreram simultaneamente. Soma-se a isso a ausência de um relógio global e a incapacidade de se confiar em um relógio local, cujo cristal de quartzo sofre desvios de frequência imprevisíveis em relação a outros computadores (**tanenbaumvansteen2007**). O relógio físico de um sistema operacional, seu *timestamp*, por assim dizer, pode estar adiantado e, em estratégias do tipo *last-write-wins*, fazer com que seu evento sobrescreva um evento posterior, cujo *timestamp* era menor (**kleppmann2017**). Também há problemas relacionados com a latência imprevisível da rede, impedindo precisão absoluta de milissegundos (**mills1991**).

Uma possível solução para esse problema é pensar em causalidade, em vez de adivinhar qual evento vem primeiro. A relação *Happened-Before* foi formulada por Leslie Lamport em 1978 (**lamport1978**), e define que um evento a aconteceu antes de um evento b, denotado $a \rightarrow b$, se:

- Ambos os eventos ocorreram no mesmo processo e a ocorreu antes de b;

- O evento a é o envio de uma mensagem e o evento b é o recebimento da mesma mensagem;

- Existe um evento c tal que $a \rightarrow c$ e $c \rightarrow b$.

Para implementar essa relação, Lamport propôs os relógios lógicos. Cada nó possui um número inteiro local (L) que é sempre incrementado na realização de um evento, garantindo a causalidade localmente (**lamport1978**). Porém, esse número precisa ser anexado ao envio de mensagens e, ao receber mensagens, o nó precisa atualizar seu relógio com o valor máximo entre o recebido e o interno, acrescido de 1. A limitação dessa solução é que, se $a \rightarrow b$, então $L(a) < L(b)$, mas o inverso não é garantido: torna-

se impossível distinguir, apenas pelos relógios, eventos causalmente relacionados de eventos concorrentes entre diferentes nodos (**fidge1988**; **mattern1989**).

Para resolver as limitações dos relógios de Lamport, foram propostos os *Vector Clocks* (VC) (**fidge1988**; **mattern1989**). Cada nó mantém um vetor de inteiros, com tamanho igual ao número de nós na rede. Cada posição do vetor representa o contador lógico de cada nó. Ao realizar um evento local, o nó incrementa sua própria posição no vetor. Ao enviar uma mensagem, o nó anexa seu vetor atual. Ao receber uma mensagem, o nó atualiza seu vetor tomando o máximo, posição a posição, entre seu vetor e o vetor recebido e, em seguida, incrementa sua própria posição, garantindo a preservação da causalidade entre eventos concorrentes. Isso garante uma ordem parcial de causalidade entre os eventos de múltiplos nodos, direta ou indiretamente:

- Se $a \rightarrow b$ pelas duas primeiras regras da relação *Happened-Before*, então B é diretamente dependente de A.
- Se $a \rightarrow c$ e $c \rightarrow b$, então B é indiretamente dependente de A.

Para comparar dois eventos, comparam-se seus vetores posição a posição. Se todas as posições de $V(a)$ são menores ou iguais às de $V(b)$ e ao menos uma é estritamente menor, então $V(a) < V(b)$, o que ocorre se, e somente se, $a \rightarrow b$. Caso nem $V(a) \leq V(b)$ nem $V(b) \leq V(a)$, os eventos são concorrentes ($a \parallel b$), situação que os relógios de Lamport não eram capazes de identificar anteriormente (**mattern1989**).

2.2.3 O Dilema da Concorrência e Consistência Eventual Forte (SEC)

Em uma arquitetura P2P descentralizada, órfã de um servidor geral agindo como uma fonte única da verdade (*Single Source of Truth*), a sincronização de dados em tempo real enfrenta o dilema da divergência de dados. Uma dada informação, alterada por dois usuários ao mesmo tempo, precisa ser sincronizada não só por ambos, mas também por toda a rede, que também pode estar alterando outros dados paralelamente. A latência e a ordenação dos fatos podem levar nodos distintos a terem versões diferentes da realidade, o que pode ser contornado com a reintrodução de um servidor central (**tanenbaumvansteen2007**). O problema é o custo com isso atrelado, isto é, a reintrodução, também, de um ponto único de falha (*Single Point of Failure*).

Os relógios lógicos vistos anteriormente permitem identificar a ordem causal e a concorrência entre eventos, mas não resolvem o conflito em si. Para garantir uma consistência forte, os nodos precisariam se coordenar, por meio de consenso ou de uma ordenação total das operações, aguardando a confirmação dos demais antes de aplicar cada alteração. Em redes com latências variáveis, essa espera se torna muito longa, destruindo a disponibilidade do sistema.

O Teorema CAP, proposto como conjectura por Eric Brewer em 2000 (**brewer2000**) e provado formalmente por Gilbert e Lynch em 2002 (**gilbertlynch2002**),

descreve o dilema da consistência em sistemas distribuídos. Ele afirma que é impossível para um sistema distribuído fornecer simultaneamente mais de duas das três garantias: consistência (C), disponibilidade (A) e tolerância a partições (P). Para o caso das redes P2P, nas quais partições de rede são inevitáveis (P), a escolha, durante uma partição, é entre a consistência forte (C) e a disponibilidade (A) (**brewer2012**). A escolha de A garante a usabilidade pelo usuário, mas permite que os nodos divirjam temporariamente. Já a escolha de C garante o uso correto da rede, mas torna a experiência lenta ou indisponível. Mesmo na ausência de partições, há uma segunda troca, entre latência (L) e consistência (C), formalizada pelo modelo PACELC (**abadi2012**), que é justamente o cenário de redes P2P com latências variáveis.

Em aplicações colaborativas em tempo real, costuma-se optar por AP, ou seja, a disponibilidade e tolerância a partições. Para lidar com a consistência, adota-se a consistência eventual (EC), que garante que todos os nodos eventualmente convergirão para o mesmo estado final, desde que não haja novas atualizações (**vogels2009**). A consistência eventual forte (SEC) vai além: quaisquer réplicas que tenham recebido o mesmo conjunto de atualizações estão imediatamente no mesmo estado, sem necessidade de reversões ou de coordenação entre os nodos (**shapiro2011**). Para isso, são utilizadas estruturas de dados específicas, como os Tipos de Dados Replicados Livres de Conflitos (CRDTs) (**shapiro2011**), que permitem a resolução de conflitos de forma descentralizada e sem a necessidade de um servidor central.

2.3 ESTRUTURAS DE DADOS PARA RESOLUÇÃO DESCENTRALIZADA DE CONFLITOS (CRDTS)

2.3.1 Semirreticulados Superiores (*Join-Semilattices*) e Propriedades Matemáticas

Lidar com divergências de estado em sistemas distribuídos sempre foi um dos principais impasses da área. No início dos anos 2000, essa comunidade provou o Teorema CAP (**brewer2000**; **gilbertlynch2002**) e, com isso, a impossibilidade de se ter consistência forte, disponibilidade e tolerância a partições simultaneamente. O OT era direcionado para sistemas centralizados e quebrava em P2P (**oster2006**), algoritmos de consenso tradicionais como Paxos (**lamport1998**) e *locks* forçavam os nós a conversarem entre si, prejudicando a disponibilidade e a latência das soluções. A resolução veio, novamente, mudando a estratégia de pensamento.

Em vez de forçar a rede a concordar sobre a ordem de mensagens, a ideia era atribuir na própria estrutura de dados a capacidade e regras de fusão fundamentadas em propriedades algébricas. Garantindo isso, a ordem em que as mensagens chegavam pela rede parava de importar (**shapiro2011**). As três propriedades fundamentais

são a comutatividade, associatividade e a idempotência.

Comutatividade garante que a ordem de chegada e aplicações de alterações não importa, o resultado final será o mesmo. Associatividade garante que o agrupamento de múltiplos pacotes, por latência de rede, por exemplo, não importa, o resultado final também será o mesmo. Para finalizar, idempotência garante que a aplicação de uma alteração que o nó já possui não alterará o estado final.

Essas propriedades não surgem por acaso. Nos CRDTs baseados em estado, os estados possíveis de uma réplica formam um semirreticulado superior (*join-semilattice*), ou seja, um conjunto parcialmente ordenado no qual quaisquer dois estados possuem um supremo (*least upper bound*), o menor estado que contém ambos. A operação de fusão (*merge*) de dois estados é justamente esse supremo, que, por definição, é comutativo, associativo e idempotente. Além disso, cada alteração local só pode fazer o estado "crescer" nessa ordem, nunca retroceder (**shapiro2011**).

A ideia, porém, não surgiu do zero. Algoritmos como o WOOT (**oster2006**), o Treedoc (**preguica2009**) e o Logoot (**weiss2009**) já exploravam estruturas comutativas para edição colaborativa em redes P2P. Com esses padrões matemáticos, Shapiro e sua equipe propuseram, comprovaram e criaram os Tipos de Dados Replicados Livres de Conflitos (CRDTs) (**shapiro2011**), provando que era possível alcançar a SEC (consistência eventual forte) em sistemas distribuídos descentralizados, sem a necessidade de um servidor central.

2.3.2 Abordagens de CRDTs: *State-based* (CvRDT), *Operation-based* (CmRDT) e *Delta-State*

Após a proposição de Shapiro, os CRDTs viraram objeto de estudo, assim como as primeiras redes P2P. Shapiro e sua equipe definiram duas abordagens equivalentes, já que uma consegue emular a outra (**shapiro2011**), e, anos depois, uma terceira foi proposta. A primeira, os *State-based* CRDTs, ou CvRDTs (*Convergent Replicated Data Types*), foi uma abordagem intuitiva, na qual, a cada alteração, o estado inteiro do CRDT era enviado para os demais nodos, que o fundiam com seu estado local por meio do *merge*. Algo muito resiliente, pois, graças à idempotência, a rede podia perder, duplicar ou reordenar mensagens sem prejudicar a convergência. A desvantagem clara é o uso de largura de banda, um *overhead* gigantesco a depender do tamanho do CRDT.

A segunda, os *Operation-based* CRDTs, ou CmRDTs (*Commutative Replicated Data Types*), enviava apenas a operação realizada, reduzindo o *overhead* de largura de banda. Em compensação, apenas as operações concorrentes precisavam comutar entre si. Entretanto, essa abordagem exigia que a rede entregasse todas as operações exatamente uma única vez e em ordem causal (*causal broadcast*), caso contrário, o estado final divergiria permanentemente. Isso aumentou consideravelmente a comple-

xidade na camada de rede do sistema (**shapiro2011comprehensive**).

A terceira, os *Delta-State* CRDTs, surgiu anos depois como um meio termo entre as duas abordagens anteriores. Em vez de enviar o estado inteiro (um grande peso para a rede, mas alta resiliência de consistência) ou apenas a operação (um peso baixo para a rede, mas alta complexidade de entrega), o *Delta-State* CRDT envia apenas o *delta*, ou seja, a diferença entre o estado anterior e o estado atual. Os *deltas* também são elementos do semirreticulado, podendo ser agrupados e fundidos entre si antes do envio. Isso reduz o *overhead* de largura de banda, mantendo a resiliência de consistência, pois o *delta* é aplicado no estado local do nó pelo mesmo *merge*. Ainda assim, para preservar a causalidade, é necessário um protocolo de sincronização entre os nós (*anti-entropy*), garantindo que todos eventualmente convergirão para o mesmo estado final (**almeida2018**).

Sobre essas abordagens foram construídos diversos tipos de CRDTs (**shapiro2011comprehensive**). Entre os mais comuns estão os contadores, como o G-Counter, que só pode ser incrementado, e o PN-Counter, que permite incrementos e decrementos; os registradores, como o LWW-Register (*Last-Writer-Wins Register*), que mantém o valor da escrita mais recente; os conjuntos, como o OR-Set (*Observed-Remove Set*), que permite adicionar e remover elementos de forma concorrente; e as sequências, como o RGA (**roh2011**), usadas na edição colaborativa de texto. Esses tipos podem ainda ser combinados em estruturas mais complexas, como mapas e documentos JSON, nos quais cada chave guarda outro CRDT (**kleppmannberesford2017**).

3 ESTADO DA ARTE E TRABALHOS RELACIONADOS

3.1 O PARADIGMA *LOCAL-FIRST SOFTWARE* E A SOBERANIA DOS DADOS

O modelo cliente-servidor é o paradigma histórico da internet há décadas. Em sua égide centralizada se consolidou o modelo *Cloud-First*, onde o estado principal da aplicação, os dados e outras informações dos usuários residem em servidores centralizados, controlados por grandes provedores (empresas, bancos, governos, entre outros) que agem como a única fonte de verdade daqueles dados (*Single Source of Truth*). Embora esse paradigma tenha resolvido os problemas de colaboração em tempo real e compartilhamento de dados, ele tornou as aplicações dependentes de conectividade e latência, além de colocar em risco a privacidade dos dados do usuário.

Alternativamente à abordagem centralizada, **kleppmann2019** propuseram um novo paradigma, guiado por sete ideais (velocidade, uso em múltiplos dispositivos, funcionamento *offline*, colaboração, longevidade, privacidade e controle do usuário). O *Local-First software* define um modelo não apenas com funcionalidades *offline*, mas com uma verdadeira inversão de hierarquia no armazenamento de dados. A cópia primária dos dados deve residir no computador local do usuário; o servidor, quando existe, é reduzido a um papel secundário de arquivamento, roteamento e auxílio às conexões ponto a ponto.

Conciliar a “independência da rede” com a “colaboração contínua” é uma batalha desafiadora em sistemas distribuídos, uma vez que, *offline*, os estados e ramificações divergem. **kleppmann2019** apontam os CRDTs como a base mais promissora para conciliar esses dois “ideais”, pois garantem a SEC ao mesclar (*merge*) alterações locais de forma determinística quando a rede é restabelecida, ainda que existam desafios em aberto.

O paradigma *Local-First* traz consigo uma discussão antiga sobre soberania de dados. No modelo estabelecido, o usuário tem, supostamente, controle de seus dados, mas precisa se comunicar por uma API para acessá-los, um direito temporário de acesso, em muitos casos. Caso ocorra um bloqueio de conta, o usuário perde o acesso, mas aqueles dados não são necessariamente deletados pelo provedor e, em incontáveis casos, são usados como produto para terceiros (**zuboff2019**). A célebre frase “Dados são o novo petróleo” por Clive Humby (**palmer2006**) ilustra a visão das empresas sobre esse aspecto.

Ao transferir a base de dados em sua completude para o armazenamento local, o sistema confere ao usuário a verdadeira custódia da informação. No contexto *peer-to-peer*, isso é elevado ao grau máximo de soberania: a centralização da infraestrutura é removida e a comunicação se torna direta entre as máquinas que detêm posse local desses dados, unificando a autonomia do armazenamento local com a topologia descentralizada de rede. Implementações em cima das redes P2P, voltadas à soberania

de dados, incluem as *blockchains* e a Web3.

3.2 *BLOCKCHAIN* E WEB3: CONSENSO GLOBAL VERSUS CONVERGÊNCIA SEM CONSENSO

Cada evolução das redes P2P até o presente foi marcada por diferentes aplicações novas surgindo em decorrência de um novo problema resolvido, como, por exemplo, o bittorrent após a inovação no envio em chuncks de arquivos. Neste contexto, nas décadas mais recentes, duas implementações se destacam: aplicações financeiras (bitcoin e ethereum) e aplicações em web 3.0.

Satoshi Nakamoto publicou em 2008 o artigo do Bitcoin, a meta era criar um sistema financeiro sem bancos capaz de realizar transações financeiras de modo quase anônimo. O maior desafio técnico dessa rede era o "problema do gasto duplo": se um usuário detém uma moeda e tenta enviá-la para a pessoa A e para a pessoa B ao mesmo tempo, a rede P2P precisa ter um mecanismo infalível para decidir qual transação vale e qual deve ser anulada. Para endereçar essa questão, o Bitcoin e do Ethereum usam a rede P2P para trocar mensagens (gossip), mas o estado real da rede não muda. Eles agrupam as transações em blocos e usam o poder computacional (Proof-of-Work) ou apostas financeiras (Proof-of-Stake) para garantir quem deve escrever no próximo bloco, atingindo o consenso global síncrono. Toda a rede precisa concordar com uma ordem total e estrita dos eventos. Todos os computadores do mundo processam exatamente a mesma fila de informações.

Para o caso da Web 3.0, o modelo anterior é inviável. Uma aplicação colaborativa não pode esperar segundos em uma fila de operações a cada movimento do mouse ou edição de um documento. Aqui entra a teoria os CRDTs, usados para, de forma deterministica, convergir a rede para o mesmo estado, sem a necessidade de consenso. Se o Usuário A move um marcador para a coordenada X, e o Usuário B move o mesmo marcador para a coordenada Y exatamente no mesmo milissegundo, a rede não precisa travar e fazer uma votação global para decidir qual edição prevalece. O caso colaborativo usa a rede P2P de forma otimista, o computador aceita a mudança localmente na hora (Local-First, zero latência) e, em segundo plano, envia a edição para os outros nós da rede.

ESSA SEÇÃO PRECISA DE UMA MELHORA DE COESÃO, IDEIA E ESTRUTURA! WORK IN PROGRESS.

3.3 PROTOCOLOS SOCIAIS DESCENTRALIZADOS: AT PROTOCOL E NOSTR

Outro caso de uso moderno de protocolos descentralizados são os protocolos sociais. Foram pautados em uma contraestratégia ao modelo das redes sociais da web 2.0, onde as grandes empresas de tecnologia centralizadas controlam, vendem

e espionam dados dos usuários a todo momento. De forma contrária, a indústria e a comunidade *open-source* começaram a buscar protocolos onde o usuário é dono do seu grafo social e dos seus dados.

Atualmente, essas redes sociais estão entre as maiores redes distribuídas em uso. A literatura e a documentação mostram como sincronizar milhões de pequenos eventos diários sem a utilização engessada de uma blockchain. Porém, na prática, alguns ainda dependem de servidores grandes, como o Bluesky (**kleppmann2024bluesky**). O problema que eles resolvem é o dos "jardins murados" (*walled gardens*) da Web 2.0 (como Twitter e Facebook), onde a empresa central é dona do seu grafo social e dos seus dados (**bernerslee2010**). Se o usuário for banido ou a empresa falir, ele perde tudo.

3.3.1 Nostr: Notes and Other Stuff Transmitted by Relays

O Nostr é um protocolo minimalista que concentra o poder no cliente local do usuário em detrimento dos grandes servidores centrais, não é necessário um *login*, criação de conta ou exposição de *e-mail*/senha a terceiros. A identidade do usuário é um par de chaves criptográficas (pública e privada), semelhante ao modelo de identidade das redes P2P. Porém, o Nostr não é P2P: os clientes se conectam a servidores (*relays*) que não conversam entre si. Esses servidores se tornam "burros" agindo apenas como espalhador de mensagens criadas e assinadas criptograficamente nos nós "inteligentes" (computador, aplicativo do celular, televisão inteligente) locais dos usuários. Se algum desses servidores bloquear o usuário por quaisquer razões, seus dados não estão perdidos, basta se conectar a outro servidor para espalhar suas mensagens. Como a chave criptográfica pertence ao usuário, seus seguidores o encontrarão na nova rota, desde que saibam em quais *relays* ele publica (**nip65**). Os dados e a identidade pertencem puramente ao nó local do usuário (**nip01**).

3.3.2 AT Protocol: Authenticated Transfer Protocol (Bluesky)

O Authenticated Transfer Protocol foi desenvolvido pela equipe do Bluesky (projeto iniciado pelo Twitter em 2019 e independente desde 2021) com participação de pesquisadores de sistemas distribuídos, como Martin Kleppmann (**kleppmann2024bluesky**). Surge como o protocolo por trás da rede social Bluesky, focado na portabilidade de contas e na separação entre quem guarda os dados e quem mostra o *feed* para o usuário. Alternativamente a um banco de dados centralizado de um grande provedor, usuários têm o direito de hospedar o seu próprio PDS (*Personal Data Server*) em um servidor próprio, que precisa ficar sempre *online*, para guardar seu estado pessoal.

Para organizar o repositório de cada usuário, são empregadas *Merkle Search Trees* (**auvolattaiani2019**). Elas servem de base para CRDTs de estado e permitem

que os indexadores da rede comparem o estado de dois servidores rapidamente e troquem apenas os deltas (as diferenças ou novas postagens), sem precisar copiar o banco de dados inteiro. Como o estado dos dados está separado do restante, os usuários têm o direito a escolher qual algoritmo irá organizar o seu *feed* personalizado, resultando em uma forma de soberania algorítmica (**kleppmann2024bluesky**).

3.4 LIBP2P: A MODULAR NETWORKING STACK FOR PEER-TO-PEER APPLICATIONS

3.4.1 Contexto e motivação

Historicamente, aplicações *peer-to-peer* (como Napster ou BitTorrent) possuíam camadas de rede monolíticas, com protocolos de transporte e mecanismos de descoberta de pares rigidamente acoplados à lógica de suas aplicações. Isso criava sistemas inflexíveis, incapazes de se comunicar entre nodos usando diferentes meios de transporte ou ambientes (navegador, servidor, smartphone).

A motivação por trás da libp2p foi criar uma *stack* de rede modular agnóstica. A internet é um ambiente fora de controle, repleto de *firewalls*, roteadores NAT e dispositivos operando em camadas e contextos distintos. A libp2p foi imaginada para integrar esses diferentes sistemas, independentemente de seus protocolos, redes, sistemas operacionais e ambientes distintos (**libp2p**). Para isso, ela é composta por uma coleção de módulos que podem ser empilhados e usados em conjunto, conforme a necessidade da aplicação.

3.4.2 Funcionamento

Diferente do modelo cliente-servidor tradicional, onde os nós são identificados por endereços de IP, na libp2p cada nó é identificado por um PeerID, derivado da sua chave criptográfica pública (**libp2ppeerids**). O endereço de IP é tratado apenas como uma rota temporária (multiaddr) para chegar àquela identidade (**multiaddr**).

A biblioteca suporta diversos meios de transporte, como TCP, QUIC (sobre UDP), WebRTC (para *browsers*) e WebSockets (**libp2pspecs**). Uma vez que dois nós tenham estabelecido uma conexão segura, é permitido que múltiplos subprotocolos paralelos rodem sobre essa mesma conexão (multiplexação), sem interferir uns nos outros (**libp2pconnections**). A ferramenta também resolve o clássico problema de encontrar nós, através de módulos intercambiáveis (**libp2pspecs**).

Já para o problema de NAT, a libp2p oferece três módulos: o AutoNAT, que detecta se o nó está atrás de um NAT (**libp2pautonat**); o Circuit Relay, que usa um outro nó como intermediário da conexão (**libp2prelay**); e o DCUtR, que tenta transformar essa conexão intermediada em uma conexão direta, através de *hole punching* (**libp2pdcutr**).

3.4.3 Implementações e Casos de Uso

A forte adoção da libp2p validou o modelo modular para toda uma indústria. Ela atua como infraestrutura de transporte em grandes redes como a camada de consenso do Ethereum (**ethp2p**) e o IPFS (**libp2p**).

3.5 IROH

3.5.1 Contexto e motivação

Embora a libp2p e o IPFS (para sistemas de arquivos) tenham se estabelecido como padrões na indústria para redes *peer-to-peer* e Web3, essas ferramentas ainda são consideradas, para algumas aplicações, complexas. Como alternativa, surgiu uma segunda opção, que evoluiu conceitualmente e respondeu a gargalos da libp2p, com foco em velocidade, simplicidade e dispositivos móveis. O acoplamento entre os componentes da libp2p (DHT, *pubsub*, negociação de transporte) torna o sistema difícil de configurar e de entender (**irohfaq**).

O Iroh nasceu como um projeto de reescrita do IPFS na linguagem Rust para torná-lo mais rápido. Contudo, para atingir as métricas de desempenho e confiabilidade desejadas, os desenvolvedores perceberam que precisariam quebrar a compatibilidade com as implementações existentes do IPFS (**irohnewdirection**). A motivação do Iroh é fornecer conexões P2P diretas entre dispositivos, identificados por chave pública, com a menor latência possível e sem a complexidade de configuração da camada de rede (**iroh**).

3.5.2 Funcionamento

O Iroh foi construído nativamente sobre o protocolo QUIC (**rfc9000**), a base do HTTP/3 (**rfc9114**), descartando o TCP (**iroh**). O QUIC oferece estabelecimento rápido de conexão, criptografia nativa (TLS 1.3) e ausência do bloqueio de cabeça de fila (*head-of-line blocking*) entre *streams*, tornando a troca de mensagens mais rápida, mesmo em redes instáveis, como 4G e 5G.

Diferente de protocolos antigos que usam SHA-256, o iroh-blobs, protocolo de transferência de dados do Iroh, utiliza o algoritmo de hash BLAKE3 (**irohblobs**). Por ser internamente uma árvore de Merkle, ele permite que os dados sejam processados de forma paralelizada e verificados durante a transmissão (*verified streaming*), aproveitando a velocidade máxima do hardware (**blake3**).

Ele resolve problemas com NAT (roteadores bloqueando conexões de fora), um dos principais de aplicações P2P, com servidores de repasse (*relays*), originalmente chamados de DERP (*Designated Encrypted Relay for Packets*) (**irohchangelog**). A conexão começa passando pelo *relay*, por onde os nós trocam informações de rede e tentam se conectar diretamente (*hole punching*). Se conseguirem, o *relay* sai do

caminho; se os *firewalls* forem muito rígidos, o tráfego continua passando por ele. O *relay* não consegue ler os dados, pois a conexão é criptografada de ponta a ponta (**irohrelay**). O usuário não percebe a diferença, pois ela é abstraída pelo Iroh.

Sobre essa camada de conexão, o Iroh oferece protocolos separados, como o iroh-docs, para sincronização de estados (**irohprotocols**). Ele permite que múltiplos nós editem uma estrutura de dados de chave-valor concorrentemente e a rede cuida de sincronizar as atualizações, através de uma técnica chamada *range-based set reconciliation* (**irohdocs**; **meyer2022**).

3.5.3 Implementações e Casos de Uso

O Iroh já é utilizado em aplicações reais. O Delta Chat, um aplicativo de mensagens descentralizado (**deltachat**), usa o Iroh em duas funcionalidades: para transferir o *backup* de um dispositivo para outro através de um QR code (**deltachattransfer**), e para criar canais de comunicação em tempo real entre os aplicativos webxdc que rodam dentro das conversas, através do iroh-gossip (**deltachatpeerchannels**). Nos dois casos, os dispositivos se conectam diretamente, sem que os dados passem pelo servidor de mensagens.

Já a Nous Research utiliza o Iroh na Psyche, uma rede para treinamento distribuído de modelos de IA pela internet. Os nós trocam metadados sobre os resultados do treinamento através do iroh-gossip e baixam os resultados em si através do iroh-blobs. Segundo os autores, o Iroh consegue estabelecer conexões diretas em cerca de 90% dos casos, contra 70% da libp2p, e migra a conexão entre redes sem interrompê-la, graças ao QUIC (**nouspsyche**).

Por fim, os desenvolvedores do Iroh mantêm uma integração de exemplo com o Automerge, onde o Iroh é usado como camada de rede para sincronizar documentos CRDT entre os nós (**irohautomerge**).

3.6 YJS

3.6.1 Contexto e motivação

A prova acadêmica dos CRDTs resolveram os problemas matemáticos para sincronização de estados, mas consumiam muita memória e CPU no processo. A serialização de grandes JSONs travava navegadores mais fracos, a depender do tamanho dos documentos serializados. O Yjs foi criado por Kevin Jahns para preencher a lacuna entre a teoria acadêmica e alta performance, implemetando uma biblioteca capaz de suportar a edição colaborativa em grande escala sem gargalos de memória e CPU.

Yjs implementa o algoritmo YATA, anteriormente desenhado para evitar o embaralhamento de textos quando usuários diferentes editam os mesmos valores. Ele modela o documento colaborativo como uma lista duplamente encadeada, em vez de árvores.

O Run-Length Encoding (RLE) usa codifiação binária compacta para condensar milhares de operações em vetor de poucos bytes. O Yjs é focado nos dados e agnóstico em relação à rede, permitindo acoplar os deltas binários em WebRTC, WebSockets ou malhas P2P.

3.6.2 Funcionamento

O Yjs não faz suposições sobre a rede usada nas implementações. Paradigmas como cliente-servidor são tão usáveis quanto *peer-to-peer*: desde que as mudanças sejam comunicadas, ocorrerá a sincronização dos estados. Isso ocorre independentemente de a comunicação ser feita por WebSockets, WebRTC (*browsers*) ou outros *providers* (**yjs**). Ele suporta edição *offline*, o que permite seguir a filosofia Local-First, onde as alterações são aplicadas primeiramente no local e depois comunicadas aos outros nodos. Eles, por sua vez, aplicam essas alterações, que são mescladas automaticamente pelos CRDTs, abstraídos como estruturas de dados convencionais do JavaScript.

Internamente, todos os tipos compartilhados (*shared types*) são listas. Isso significa que um único algoritmo, o YATA (**nicolaescu2016**), resolve os conflitos de todos os tipos. Cada item tem um ID, formado pelo identificador do cliente e por um relógio de Lamport, e guarda os vizinhos do momento em que foi inserido. Com isso, dois usuários que inserem no mesmo ponto chegam à mesma ordem final. Um item deletado não desaparece, ele vira um *tombstone* no conjunto de deleções (*delete set*). Para que os metadados não cresçam *ad infinitum*, o Yjs junta em um único item os caracteres digitados em sequência pelo mesmo usuário e descarta o conteúdo dos itens deletados. Em um documento real, com 182 mil inserções e 77 mil deleções, o conjunto de deleções ocupou apenas 4,5 KB (**yjsinternals**).

3.6.3 Implementações e Casos de Uso

Usado por diversas aplicações (**yjs**), o Yjs é utilizado tanto em aplicações cliente-servidor quanto *peer-to-peer*. Entre os usuários mais conhecidos estão o JupyterLab, para notebooks colaborativos, e o JupyterGIS, um editor colaborativo de sistemas de informação geográfica (**yjs**; **jupytergis**). O Room.sh, uma ferramenta de reuniões com funcionalidades embutidas de desenho, edição e codificação, faz uso do *framework* para sincronizar seus vários estados entre múltiplos usuários. Outras aplicações diferentes também contam com uso do Yjs, como o Dynaboard, um construtor colaborativo de aplicações web, e o LegendKeeper, uma ferramenta colaborativa para planejamento de campanhas de RPG (**yjs**).

3.7 AUTOMERGE

3.7.1 Contexto e motivação

A motivação do Automerge era permitir aos programadores a criação de softwares Local-First manipulando estruturas de dados familiares (listas, *strings*, objetos JSON) sem precisar lidar com os problemas complexos de sistemas distribuídos (**automerge**). Nascido com foco na experiência do desenvolvedor e na preservação de histórico, o Automerge funciona como um "Git para estruturas de dados" (**automerge**). Foi criado pelo laboratório Ink & Switch, o mesmo que publicou o artigo fundador do Local-First, com participação de Martin Kleppmann, um dos autores desse artigo, e é baseado na pesquisa sobre CRDTs para JSON (**kleppmann2019**; **kleppmannberesford2017**).

3.7.2 Funcionamento

Enquanto o Yjs representa internamente todos os tipos como listas, com foco em desempenho, o Automerge modela o documento como uma árvore JSON aninhada (**kleppmannberesford2017**). Na versão 3.0, o Automerge passou a usar o seu formato colunar comprimido também durante a execução, e não apenas no armazenamento, reduzindo o uso de memória em mais de 10 vezes (**automerge3**). Ele salva o histórico completo das mutações compactando colunas de dados semelhantes (**automerge3**). Já a sincronização envia apenas as alterações que o outro nó ainda não possui (**automerge**).

Assim como no Git, cada alteração feita em um documento é registrada como uma *change*, parecida com um *commit*. Cada *change* é identificada pelo *hash* SHA-256 do seu conteúdo e aponta para as *changes* das quais depende, formando um grafo acíclico dirigido (DAG). As *changes* mais recentes desse grafo, que ainda não possuem sucessoras, são chamadas de *heads* (**automergespec**). Dentro de cada *change*, toda operação recebe um ID formado por um contador e pelo identificador da réplica (*actor ID*), ou seja, um relógio de Lamport. Isso impõe uma ordem total entre as operações: vence o maior contador e, em caso de empate, o maior *actor ID* (**automergespec**).

Na maioria dos casos, a mesclagem é direta. Alterações em objetos ou propriedades diferentes são simplesmente combinadas. Em listas e textos, todas as inserções e deleções são preservadas, e inserções concorrentes na mesma posição recebem a mesma ordem em todos os nós (**automergeconflicts**). Já para valores numéricos, o Automerge oferece o tipo `Counter`, que soma incrementos concorrentes: se dois usuários incrementam um valor 3 ao mesmo tempo, o resultado é 5, e não 4 (**automergecounters**).

O único caso sem resolução automática é quando dois usuários alteram a mesma propriedade do mesmo objeto. Nesse caso, o Automerge escolhe um dos valores como vencedor, de forma determinística e igual em todos os nós, sem depender do

relógio físico das máquinas. Os valores perdedores não são descartados: eles ficam acessíveis pela função `getConflicts()`, permitindo que a aplicação mostre o conflito ao usuário (**automergeconflicts**).

A sincronização entre dois nós é baseada no trabalho de **kleppmannhoward2020**. Cada nó guarda o que sabe sobre as *heads* do outro e, a cada mensagem trocada, envia apenas as *changes* que o outro ainda não possui, até que os dois tenham o mesmo histórico (**automergesync**). Por cima disso, o Automerge Repo cuida da parte de infraestrutura: ele salva os documentos localmente através de adaptadores de armazenamento, como o IndexedDB no navegador ou o sistema de arquivos, e os distribui para outros nós através de zero ou mais adaptadores de rede, como o WebSocket (**automergerepo**).

3.7.3 Implementações e Casos de Uso

Os casos de uso podem variar, mas ele se estende para todas as aplicações Local-First ou que requerem um histórico de edições, como no Git. Aplicações Local-First exigem armazenamento local robusto e controle de versão, como gerenciadores de conhecimento, ferramentas de produtividade *offline* e outros tipos de sistemas onde os usuários necessitam viajar entre os históricos passados de alterações.

Os primeiros casos de uso foram os protótipos do próprio Ink & Switch, apresentados no artigo sobre Local-First (**kleppmann2019**): o Trellis, um quadro Kanban inspirado no Trello, que experimentou uma interface de “viagem no tempo” para navegar pelos estados anteriores do documento; o Pixelpusher, um editor colaborativo de *pixel art*; e o PushPin, um mural colaborativo de mídias mistas, parecido com o Miro, considerado pelos autores o mais completo dos três. Além disso, como visto na seção sobre o Iroh, existe uma integração de exemplo que usa o Iroh como camada de rede para sincronizar documentos do Automerge (**irohautomerge**).

3.8 ORBITDB E MERKLE-CRDTS

3.8.1 Contexto e motivação

A libp2p é usada como a camada de rede do protocolo IPFS (**libp2p**). O IPFS é excelente para distribuir arquivos estáticos de forma imutável, mas não funciona bem para arquivos que mudam dinamicamente, como em um banco de dados, pois os arquivos são endereçados pelo *hash* do seu conteúdo e qualquer alteração gera um novo identificador (**benet2014**). O OrbitDB foi criado para ser um banco de dados distribuído e sem servidor, voltado para aplicações P2P, Web3 e Local-First. Ele une o armazenamento P2P do IPFS com a consistência eventual dos CRDTs, eliminando completamente a figura do servidor de banco de dados tradicional (**orbitdb**).

3.8.2 Funcionamento

Todo banco de dados no OrbitDB é formado por um registro de operações (*oplog*), um *log* imutável onde novas operações são apenas adicionadas ao final (*append-only*). O estado do banco é calculado a partir dessas operações. Sobre o *oplog*, o OrbitDB oferece diferentes tipos de banco de dados: *events* (um *log* de eventos), *documents* (documentos JSON indexados por uma chave), *keyvalue* e *keyvalue-indexed* (chave-valor) (**orbitdb**).

O *oplog* é formalmente um Merkle-CRDT (**sanjuan2020**). Cada operação no banco de dados, como adicionar ou remover uma chave, vira uma entrada imutável armazenada no IPFS, identificada pelo seu *hash* criptográfico. Cada entrada referencia as entradas anteriores na ordem causal, e o banco em sua completude forma um Grafo Acíclico Dirigido (DAG), semelhante aos *commits* do Git. Como cada entrada é identificada pelo *hash* do seu conteúdo e das suas referências, nenhuma entrada pode ser alterada ou removida sem que isso seja detectado (**orbitdboplog**). A ideia central dos Merkle-CRDTs é que o próprio DAG funciona como um relógio lógico, o que simplifica a construção de CRDTs em redes onde as mensagens podem ser perdidas, duplicadas ou chegar fora de ordem (**sanjuan2020**).

A sincronização é feita através da libp2p. Ao abrir um banco, o nó se inscreve em um tópico de *pubsub* com o identificador do *log*. Os nós que já estão no tópico se conectam diretamente ao novo nó e os dois trocam as *heads* do *log*. Depois disso, a cada nova operação, o nó publica as suas novas *heads* no tópico, e os outros nós buscam as entradas que ainda não possuem. O protocolo não garante a ordem nem a entrega das mensagens, mas garante que, quando todas forem entregues, todos os nós terão o mesmo estado, ou seja, consistência eventual (**orbitdbsync**).

3.8.3 Implementações e Casos de Uso

A implementação principal do OrbitDB é em JavaScript e funciona tanto no navegador quanto no Node.js. Existe também uma implementação em Go, a go-orbit-db, desenvolvida e mantida pelo projeto Berty (**orbitdb**). O Berty é um aplicativo de mensagens *offline-first*, *peer-to-peer* e sem servidor (**berty**). No seu protocolo, chamado Wesh, as conversas acontecem em “grupos”, onde múltiplos dispositivos compartilham mensagens e metadados através do OrbitDB (**berty**).

Os Merkle-CRDTs também são usados fora do OrbitDB. O go-ds-crdt, um banco chave-valor baseado no mesmo artigo de **sanjuan2020**, replica automaticamente cada par chave-valor para todos os nós da rede, mesmo que as mensagens sejam perdidas, reordenadas ou duplicadas (**godscrdt**). Ele é usado pelo IPFS Cluster, uma ferramenta que coordena quais arquivos devem ser replicados entre vários nós do IPFS, para manter esse estado global sincronizado entre todos os nós do *cluster* (**ipfscluster**).

3.9 SINCRONIZAÇÃO COLABORATIVA DE DADOS ESPACIAIS (WEBGIS)

WORK IN PROGRESS - É PRECISO AVANÇAR MAIS PARA CONTINUAR ESSE CAPITULO

3.10 COMPARATIVO E POSICIONAMENTO DO TRABALHO

WORK IN PROGRESS - É PRECISO AVANÇAR MAIS PARA CONTINUAR ESSE CAPITULO

Tabela 1 – Comparativo entre libp2p e Iroh

Critério	**libp2p**	**Iroh**
Arquitetura	Coleção de módulos empilháveis, conforme a necessidade da aplicação	Núcleo de conexão, com protocolos separados por cima (iroh-blobs, iroh-gossip, iroh-docs)
Implementações	Várias (Go, Rust, JavaScript, entre outras), seguindo as mesmas especificações	Rust
Identidade do nó	PeerID, derivado da chave pública	Chave pública Ed25519
Transportes	TCP, QUIC, WebRTC, WebSockets, entre outros	QUIC
Segurança e multiplexação	Módulos próprios sobre a conexão (ou nativos, no QUIC)	Nativos do QUIC
Travessia de NAT	AutoNAT, Circuit Relay e DCUtR (*hole punching*)	Conexão inicia pelo *relay* e migra para direta via *hole punching*
Conexões diretas (segundo a Nous Research)	Cerca de 70%	Cerca de 90%
Descoberta de pares	Kademlia DHT, mDNS e *rendezvous*	DNS (Pkarr) e, opcionalmente, a DHT do BitTorrent; sem DHT Kademlia própria
Difusão de mensagens	GossipSub	iroh-gossip (HyParView e PlumTree)
Casos de uso	Camada de consenso do Ethereum, IPFS, OrbitDB	Delta Chat, Psyche

Fonte: Elaborado pelo autor, com base em **libp2p**, **libp2pspecs**, **iroh**, **irohfaq**, **irohaddresslookup** e **nouspsyche**.

Tabela 2 – Comparativo entre Yjs e Automerge

Critério	**Yjs**	**Automerge**
Base teórica	YATA (**nicolaescu2016**)	CRDT para JSON (**kleppmannberesford2017**)
Modelo de dados	Todos os tipos são listas internamente	Documento JSON aninhado (mapas, listas, texto, contadores)
Histórico	Descarta o conteúdo dos itens deletados	Guarda o histórico completo, como um grafo de *changes*
Foco	Edição colaborativa em tempo real e desempenho	Versionamento e aplicações Local-First
Rede	Agnóstico, via *providers*	Agnóstico, via Automerge Repo
Tempo para aplicar o histórico real (B4)	5.714 ms	14.326 ms
Tamanho final do documento (B4)	159.929 bytes	129.116 bytes

Fonte: Elaborado pelo autor, com base em **yjs**, **yjsinternals**, **automerge** e **crdtbenchmarks**.

4 CONTEXTO DO ESTUDO DE CASO E CONCEITOS

5 ARQUITETURA E DESENVOLVIMENTO DO MOTOR DE SINCRONIZAÇÃO

6 AVALIAÇÃO EXPERIMENTAL E RESULTADOS

7 CONCLUSÃO